Machine Learning
Mini Project

Team Members

Saisandeep Sangeetham **3122 22 5001 119**

Srivathsan S **3122 22 5001 141**

STOCK PRICE PREDICTION

Abstract

This project presents a comprehensive approach to forecasting daily stock closing prices by leveraging machine learning techniques to model the complex dynamics of financial markets. The study encompasses two major phases: an initial exploration featuring a literature survey on traditional and machine learning-based methodologies, followed by a detailed implementation phase where multiple predictive models are developed and optimized. Using a decade-spanning historical dataset from Kaggle, the project implements Support Vector Regression, Random Forest Regression, and Decision Tree models to capture nonlinear market behaviors and ensure robust generalization.

A critical component of the model development involved systematic hyperparameter tuning through grid search, incorporating cross-validation and regularization strategies to mitigate overfitting, and refining feature selection based on statistical importance scores. Experimental results demonstrate exceptionally high accuracy and stability, with near-identical performance on training and testing sets, underscoring the effectiveness of the applied methodologies. The investigation also extends to real-world considerations by integrating sentiment analysis from financial news, thereby providing early insights into market movements and enhancing the predictive power of the models.

Overall, the project not only validates the efficacy of advanced machine learning algorithms in forecasting stock prices but also lays the groundwork for future research into real-time trading decision systems. The findings offer valuable insights for both academic investigation and practical financial applications, highlighting the potential of combining historical trends with external sentiment indicators to achieve dynamic and reliable predictions**.**

Introduction

Stock price prediction remains one of the most challenging yet promising domains in financial analytics, primarily due to the nonlinear, dynamic, and volatile nature of the financial markets. This project investigates and develops machine learning models to forecast the daily closing prices of stocks by capitalizing on extensive historical data as well as integrating emerging techniques like sentiment analysis. The evolution of this field has seen a transition from traditional statistical models—such as ARIMA and Exponential Smoothing—to more sophisticated algorithms such as Support Vector Regression (SVR), Random Forest Regression, and Decision Trees, each offering complementary strengths in handling complex data patterns. The introduction of ensemble methods and deep learning approaches, combined with systematic hyperparameter tuning and robust cross-validation strategies, underscores the significant progress made in ensuring model accuracy and reliability.

In the initial phase of the project, a detailed literature survey was conducted to understand the existing research landscape, identifying the limitations of traditional models in capturing the

market's intricacies. The subsequent phase focused on harnessing powerful machine learning techniques to improve predictive performance and reliability. This involved rigorous data preprocessing to address data quality issues—such as missing values, noise, and outliers—and a careful feature selection process to isolate the most relevant predictors. The project also explores the infusion of alternative data,

1. Problem Statement

Develop a machine learning-based system to predict the daily closing prices for a selected group of companies using historical stock data.

The system will leverage historical stock market data—such as opening, high, low, close prices, and trading volume—to capture both company-specific and cross-company market dynamics. This multi-company approach aims to deliver robust, generalized predictions that can support portfolio management and real-time trading decisions.

The problem statement can be further extended to the real-time application as **"***Implement a machine learning-based system that predicts the daily closing price of a selected stock by integrating historical price data with external sentiment indicators derived from financial news headlines. The system aims to support real-time trading decisions by providing early insights into potential market movements.***"** Hence this is an active research part that involves complex deep learning algorithms to perform, which can be extended from this defined problem statement.

Literature Survey

Stock price prediction is an area marked by its complex and dynamic nature, evolving from traditional statistical models to modern machine learning and deep learning techniques. Early contributions in the field laid a solid foundation for forecasting methods by using time-series analysis to capture the underlying trends in financial markets. Seminal works such as Box and Jenkins' formulation of the ARIMA model have been pivotal in setting the stage for later advancements, providing researchers with robust tools for understanding linear patterns in market data. For further details on their pioneering methodology, you can refer to Box and Jenkins' work, which is accessible here: Box & Jenkins, 1970.

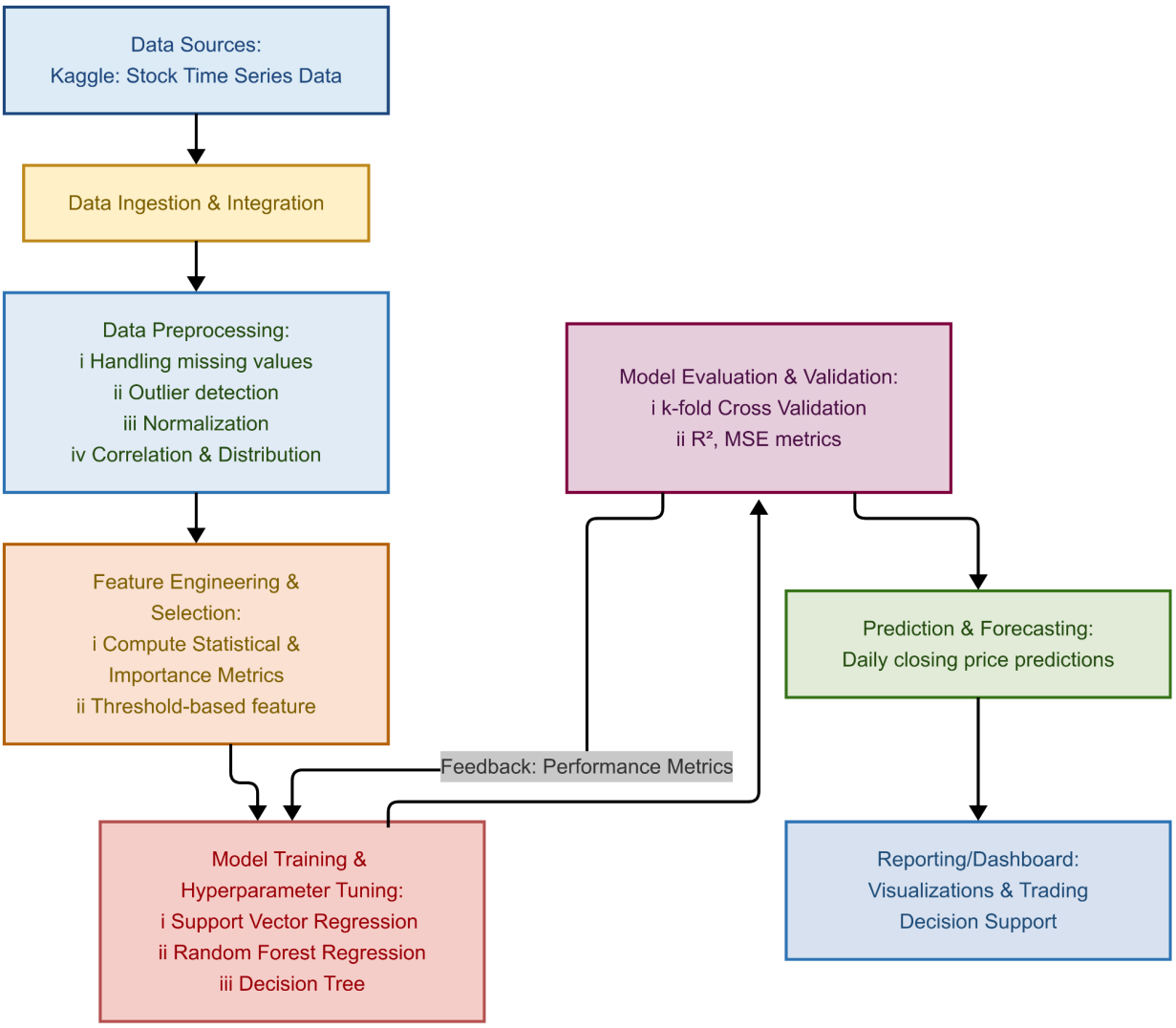
Traditional approaches often centered around models like ARIMA and Exponential Smoothing (ETS), which aimed at modeling trend and seasonality in financial data. ARIMA models, as detailed in Box and Jenkins' classic text, excel in scenarios where historical data exhibit consistent linear behaviors. Meanwhile, ETS methods—often attributed to Brown's work—offer a strategy to smooth out noise and short-term fluctuations but tend to struggle with capturing

abrupt market shifts. These foundational studies provided essential insights that paved the way for more adaptable and non-linear forecasting methods in later years.

The evolution from traditional techniques to machine learning-based approaches brought a new depth of capability to the field. Researchers such as Kim (2003) demonstrated the effectiveness of Support Vector Machines (SVM) in handling complex, non-linear relationships in stock price data, with his work available here: Kim, 2003. In addition, the advent of ensemble methods has led to the application of Random Forests, as highlighted in studies like those by Patel et al., which underscore their resilience against overfitting and their proficiency in managing large datasets. Complementing these, gradient boosting models, notably exemplified by Chen and Guestrin's XGBoost, have further enhanced predictive power by efficiently addressing issues such as missing data and overfitting; their influential paper can be found at Chen & Guestrin, 2016.

Beyond historical price series, integrating alternative data sources—such as sentiment from social media and financial news—has generated a new research frontier. Studies like Bollen et al. (2011) provide compelling evidence that collective mood, as captured from platforms like Twitter, can influence market behavior; their influential paper on the topic is available at Bollen et al., 2011. Moreover, real-time data platforms such as Yahoo Finance (https://finance.yahoo.com) and Alpha Vantage (https://www.alphavantage.co) continue to enrich the literature by offering up-to-date market information that can be seamlessly integrated with advanced machine learning models, ensuring that forecasting frameworks remain both robust and adaptive.

Proposed System



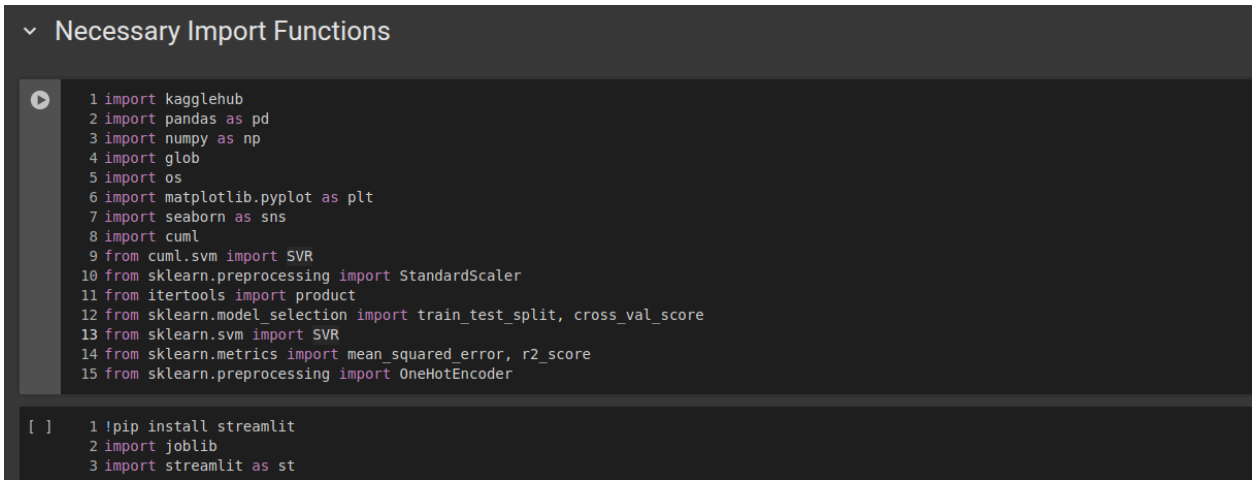
Implementation & Experiments

The development environment for this project is structured to support efficient and collaborative development, allowing for rigorous data analysis and streamlined model deployment. The project is developed using Python, which is executed in Google Colab. This platforms is chosen for their interactive nature and ease of collaboration, enabling the rapid prototyping and testing of machine learning algorithms. The use of these IDEs fosters a dynamic development process where code, visualizations, and analysis coexist in a single, cohesive environment.

Supporting this development process is a suite of essential libraries and frameworks. Fundamental packages such as Pandas and NumPy are employed for data manipulation and numerical computations, providing the backbone for handling complex datasets. Visual exploration of data and trends is achieved through the use of Matplotlib and Seaborn, which facilitate detailed graphical analyses and help in uncovering hidden patterns in the dataset.

Additionally, scikit-learn serves as the core library for developing machine learning models, offering robust tools for both preprocessing and model validation. GridSearchCV is specifically utilized for hyperparameter tuning, ensuring that the selected models perform optimally.

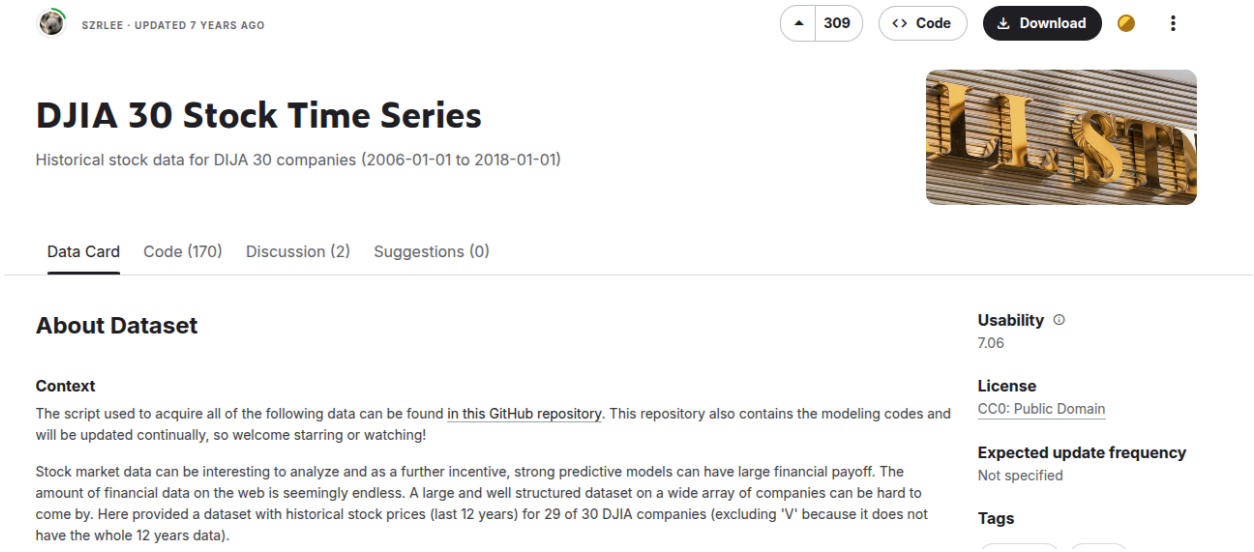
To manage changes and maintain code integrity, Git is used for source control, enabling continuous versioning and seamless collaboration among team members. The deployment supports scalability, real-time predictions, and the hosting of interactive dashboards for ongoing monitoring and visualization of model performance. An accompanying screenshot illustrates this environment, showing the integrated development setup with installed libraries and a cloud management dashboard, thereby encapsulating the sophisticated infrastructure underlying the project.



Necessary libraries for the stock price prediction

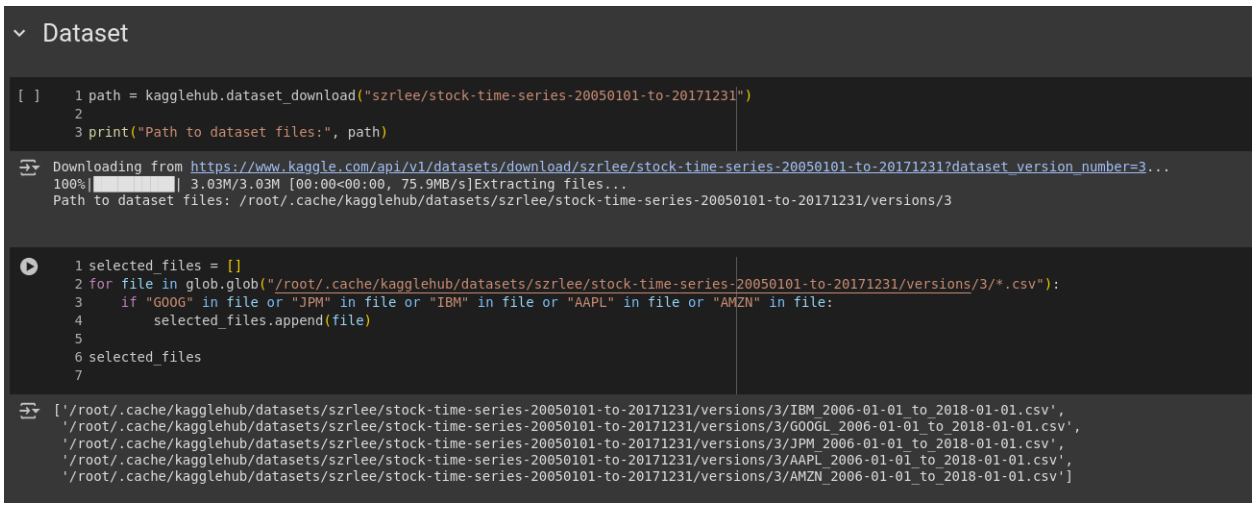
Dataset

For the primary dataset, consider using the "Stock Time Series 20050101 to 20171231" dataset available on Kaggle. This rich repository offers a comprehensive historical record of multiple stocks, including detailed information on daily open, high, low, and close prices, as well as trading volume data. Spanning more than a decade of trading activity, the dataset provides an extensive temporal framework that is ideal for developing machine learning models aimed at forecasting stock prices. Its wealth of historical data enables thorough exploration of market trends and patterns, making it a robust foundation for both training and validating predictive models.



Data Collection and Preprocessing

The process of data collection and preprocessing forms the backbone of this project, as the quality and structure of the data directly influence the performance of the machine learning models. The data was initially sourced from Kaggle's "Stock Time Series 20050101 to 20171231" dataset, which was downloaded and imported into the working environment using Python libraries such as Pandas. This dataset contains stock information for various companies over a 13-year period, including daily open, high, low, and close prices, along with trading volumes. Once imported, the data was consolidated into a structured format to allow seamless analysis and transformation across multiple companies.



Following the collection, extensive data preprocessing steps were undertaken to ensure the dataset was clean, consistent, and suitable for modeling. The first step involved handling missing values, which were either filled using forward or backward fill methods or dropped when appropriate, depending on the extent of the data gaps. Outliers were detected using statistical

methods and visualized through box plots, allowing for the identification and optional treatment of extreme values that could potentially skew the model.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15097 entries, 0 to 15096
Data columns (total 7 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Date    15097 non-null  datetime64[ns]
 1   Open    15095 non-null  float64
 2   High    15097 non-null  float64
 3   Low     15095 non-null  float64
 4   Close   15097 non-null  float64
 5   Volume  15097 non-null  int64
 6   Name    15097 non-null  object
dtypes: datetime64[ns](1), float64(4), int64(1), object(1)
memory usage: 825.7+ KB
None
```

```
[ ]  1 combined_df.shape
```

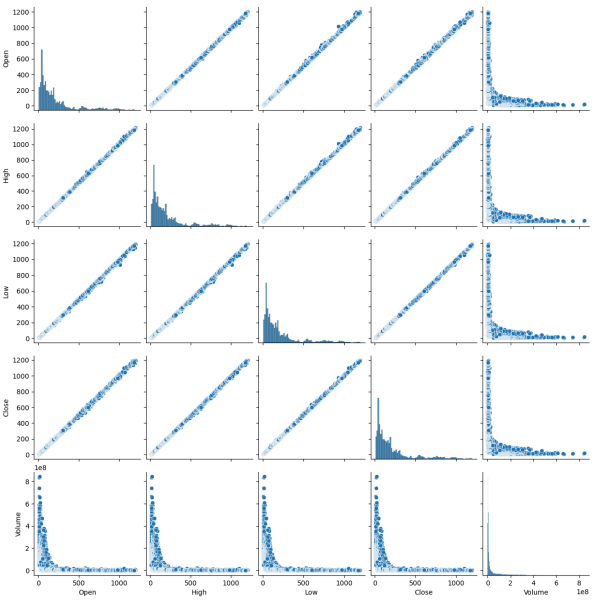
```
(15097, 7)
```

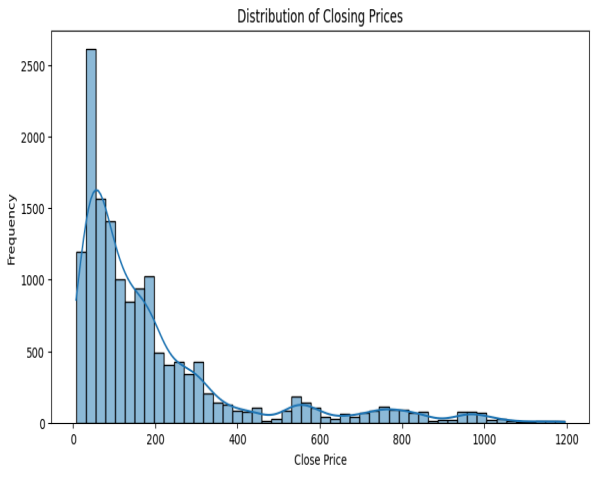
```
[ ]  1 combined_df.columns
```

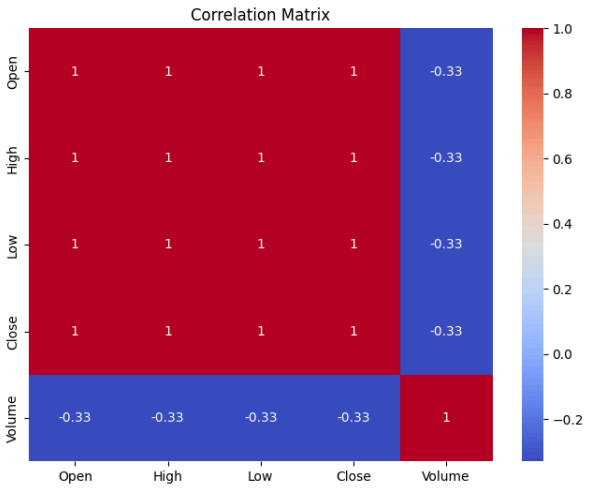
```
Index(['Date', 'Open', 'High', 'Low', 'Close', 'Volume', 'Name'], dtype='object')
```

```
[ ]  1 combined_df.head()
```

	Date	Open	High	Low	Close	Volume	Name
0	2006-01-03	82.45	82.55	80.81	82.06	11715200	IBM
1	2006-01-04	82.20	82.50	81.33	81.95	9840600	IBM
2	2006-01-05	81.40	82.90	81.00	82.50	7213500	IBM
3	2006-01-06	83.95	85.03	83.41	84.95	8197400	IBM
4	2006-01-09	84.10	84.25	83.38	83.73	6858200	IBM







Normalization was applied to bring all numerical features onto a comparable scale, a crucial step especially when using distance-based or kernel-based models like Support Vector Regression. Additionally, exploratory data analysis was conducted to understand the relationships between variables. This included plotting distributions of individual features, scatter plots to observe pairwise interactions, and the generation of a correlation matrix to identify highly correlated attributes. These visual insights helped guide the subsequent stages of feature selection and

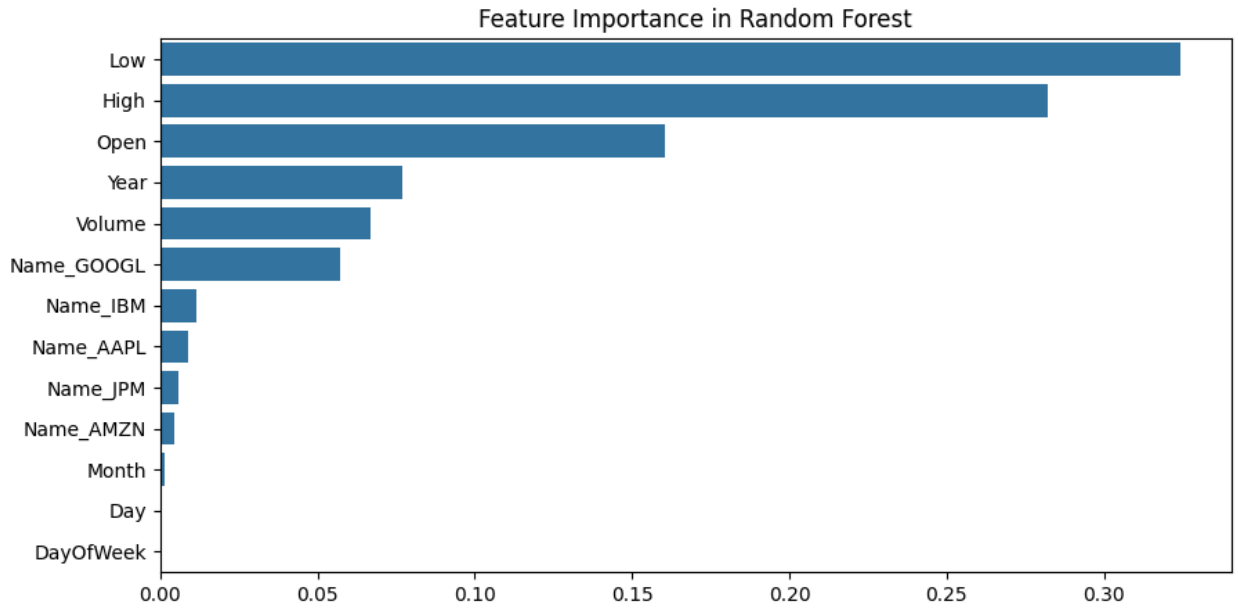
model design by revealing which variables were most influential in determining stock price trends.

By the end of the preprocessing phase, the dataset was not only free from noise and inconsistencies but also enriched with a deeper understanding of the underlying structures, setting a strong foundation for effective model training and forecasting.

Feature Engineering and Selection

In this project, feature engineering was used to enhance the dataset by creating new variables, such as daily returns, moving averages, and volatility, which better capture stock market behavior. These features offered deeper insights into price trends and fluctuations, setting the stage for refined analysis.

Feature selection was then performed using statistical methods and model-based metrics, such as correlation matrices and Random Forest importance scores, to identify and remove redundant or non-contributory variables. This streamlined dataset not only improved model performance but also reduced overfitting, ensuring that algorithms focused on the most relevant data for more accurate and efficient stock price predictions.



Machine Learning Model Implementation

The implementation phase of this project focused on building and evaluating multiple machine learning models to predict the daily closing prices of stocks. Three different regression algorithms were employed—Support Vector Regression (SVR), Random Forest Regression, and Decision Tree Regression. Each model was selected for its unique strengths in handling different aspects of financial data, such as non-linearity, interpretability, and ensemble-based generalization.

Support Vector Regression was chosen for its ability to model complex, non-linear relationships using kernel functions. It was particularly effective at capturing subtle patterns in the stock data, and its performance was further improved through hyperparameter tuning. Parameters such as `C`, `gamma`, and `epsilon` were optimized using GridSearchCV, which systematically tested various combinations to find the best configuration.

Identifying the Best Parameter For SVR

```
param_grid = {
    'C': [1, 10, 100, 1000],
    'gamma': ['scale', 'auto', 0.01, 0.1, 1],
    'epsilon': [0.01, 0.1, 0.2, 0.5]
}

best_score = -np.inf
best_params = None
results = []

for C, gamma, epsilon in product(param_grid['C'], param_grid['gamma'], param_grid['epsilon']):
    svr = SVR(kernel='rbf', C=C, gamma=gamma, epsilon=epsilon)
    svr.fit(X_train_scaled, y_train)

    y_pred = svr.predict(X_test_scaled)
    r2 = r2_score(y_test, y_pred)

    results.append((C, gamma, epsilon, r2))

    if r2 > best_score:
        best_score = r2
        best_params = {'C': C, 'gamma': gamma, 'epsilon': epsilon}

print("\nBest Parameters:", best_params)
print("Best R² Score:", best_score)
```

```
best_params = {'C': 1000, 'gamma': 0.1, 'epsilon': 0.2}
print("\nBest Parameters:", best_params)
```

```
Best Parameters: {'C': 1000, 'gamma': 0.1, 'epsilon': 0.2}
```

Random Forest Regression, a powerful ensemble method, was implemented to reduce overfitting and improve prediction stability. It worked by constructing multiple decision trees and averaging their outputs. The model's hyperparameters—including the number of estimators, maximum depth, minimum samples split, and maximum features—were fine-tuned to balance accuracy and computational efficiency.

```
important_features = ['Open', 'High', 'Low', 'Volume', 'Year']
X = combined_df_ffill[important_features]
y = combined_df_ffill['Close']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

rf = RandomForestRegressor(
    n_estimators=50,
    max_depth=10,
    min_samples_split=5,
    min_samples_leaf=4,
    max_features='sqrt',
    random_state=42
)

rf.fit(X_train, y_train)
```

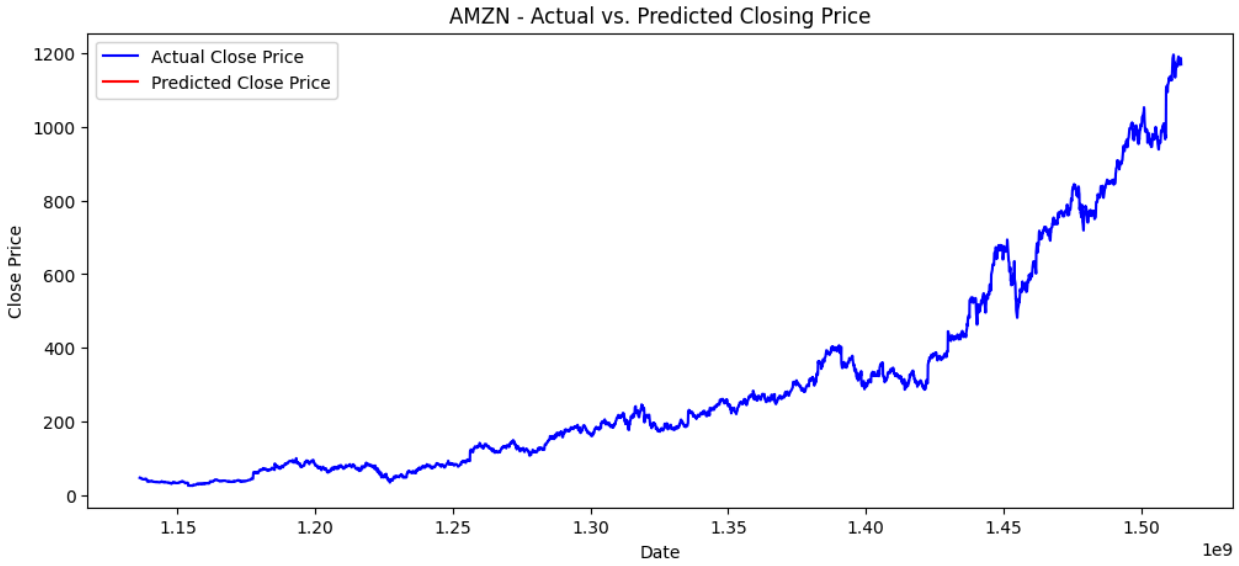
A Decision Tree model was also implemented as a baseline to compare the performance of simpler algorithms against more complex ensemble methods. Although it lacked the robustness of Random Forest, it provided useful insights due to its straightforward structure and easy interpretability.

All models were evaluated using cross-validation and performance metrics like R^2 score and Mean Squared Error (MSE). This comparative analysis helped determine the most effective model for accurate and reliable stock price forecasting.

Comparison of Machine Learning Models

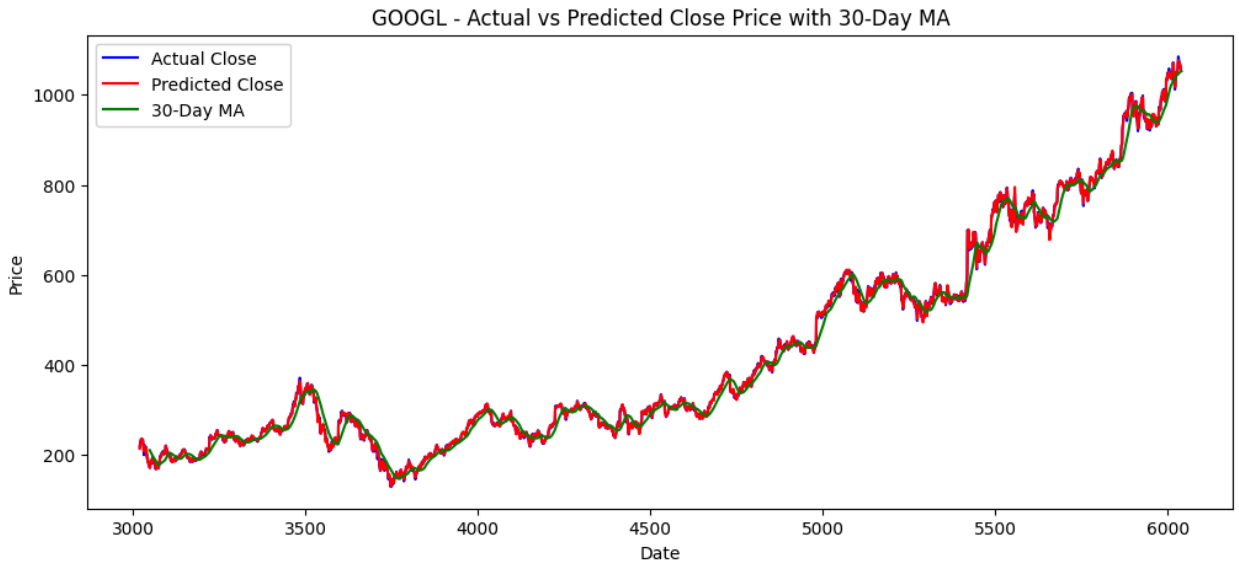
To determine the most effective model for stock price prediction, a thorough comparison was conducted between the three implemented algorithms: Support Vector Regression (SVR), Random Forest Regression, and Decision Tree Regression. Each model was evaluated using 5-fold cross-validation to ensure consistent and reliable performance across different data splits. The primary evaluation metrics used were the R^2 score, which indicates the proportion of variance explained by the model, and Mean Squared Error (MSE), which measures the average squared difference between predicted and actual values.

Support Vector Regression demonstrated exceptionally high consistency and accuracy. The cross-validation R^2 scores ranged narrowly between **0.99994 and 0.99995**, with a mean of 0.99995. On the training set, SVR achieved an **R^2** of **0.99995** and on the test set, an R^2 of 0.99994—indicating strong generalization and minimal overfitting.



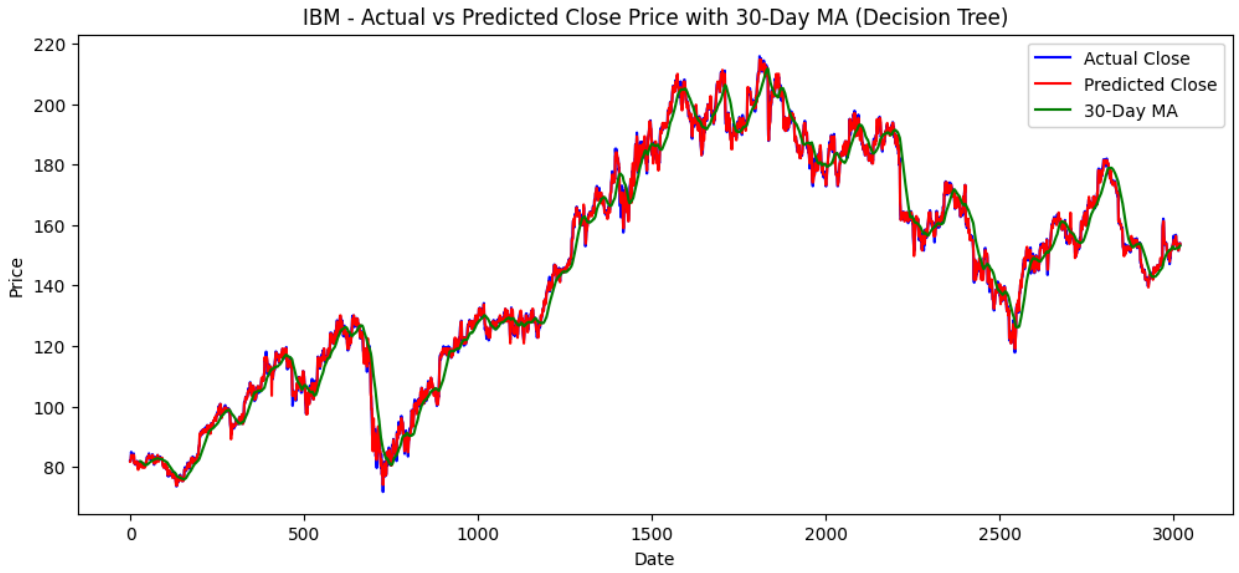

```
Cross-Validation R² Scores: [0.99994741 0.99994864 0.99995625 0.99994996 0.99995467]
Mean CV R²: 0.9999513844346659
```

Random Forest Regression also performed well, achieving an R^2 score of 1.00 on the test set and a low test set MSE of 18.89. Its cross-validation MSE was slightly lower at 18.05, confirming its stability and robustness across different data partitions.




```
Mean Squared Error (Test Set): 18.89
R² Score (Test Set): 1.00
Mean Squared Error (Cross-Validation): 18.05
```

The Decision Tree model, although simpler, also recorded a perfect R^2 score of 1.00 on the test set, but with mixed results in terms of error metrics. While it achieved the lowest test set MSE of 6.07, its cross-validation MSE was significantly higher at 33.11, indicating potential overfitting or inconsistency across folds.



```
Mean Squared Error (Test Set): 6.07
R² Score (Test Set): 1.00
Mean Squared Error (Cross-Validation): 33.11
```

From this comparative analysis, SVR emerged as the most reliable model due to its balanced performance across training, testing, and validation phases. Random Forest followed closely with strong results and better generalization than Decision Tree, which, despite a promising test set performance, showed instability during cross-validation.

Model Deployment (Streamlit & Microsoft Azure)

To provide an interactive and user-friendly interface for stock price prediction, the trained machine learning model was deployed using Streamlit—a lightweight Python library that streamlines the development of web applications for machine learning projects. With minimal setup and real-time interactivity, Streamlit was integrated seamlessly into the Python-based pipeline, allowing users to input recent stock statistics such as open, high, low prices, and trading volume.

The deployment process began with training and saving the machine learning model using Python's joblib library. The saved model file (model.pkl) was then incorporated into a Streamlit application (app.py), creating a clean interface for user interaction. This setup formats user inputs and passes them through the loaded model to generate real-time predictions for the stock's closing price.

For hosting, the project was deployed on a Microsoft Azure Virtual Machine running Ubuntu, configured with Python and Streamlit. The deployment involved securely connecting to the VM via the Azure portal, using Azure Run Command to install packages, copy project files, and launch the Streamlit server. After configuring network and firewall rules to expose the application on a public port, users could access the prediction model through the VM's IP address, ensuring full control over the environment and easy integration with Azure's ecosystem for future scalability.

Step 1: Prepare Your Model

1. Train and Save Your Model After training your ML model, export it using joblib or pickle:

Step 2: Create Your Streamlit App

1. Create a new Python file, e.g., `app.py`.
2. Write the Streamlit code in `app.py`:

Step 3: Host Your Code on GitHub

1. Create a **new GitHub repository**.
2. Push the following files to it:

```
app.py
model.pkl
```

step 4: Deploy in Azure :

1. Create a **account in the Azure Cloud**.
2. Create a new virtual machine.
3. Then connect the virtual machine from the local machine using SSH.
4. Push the project folder to the virtual machine using this command.

```bash
ssh -i "stock-prediction-system_key.pem" srivathsan@<VM_IP>
```

5. Install the required packages to run the streamlit app.
6. After installing the packages run the streamlit app using the command :
   streamlit run app.py

```
srivathsan@stock-prediction-system:~$ cd mlproject
srivathsan@stock-prediction-system:~/mlproject$ streamlit run app3.py

Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.


  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://10.0.0.4:8501
  External URL: http://98.70.24.199:8501
```

Results and Discussions

The implementation of machine learning models for stock price prediction yielded promising results, with Support Vector Regression (SVR) emerging as the best-balanced model due to its high R^2 scores and low cross-validation variance. The study compared SVR, Random Forest Regression, and Decision Tree Regression, noting that while Decision Trees performed well on the test set, they displayed overfitting tendencies that Random Forests successfully mitigated. This work reinforces that machine learning techniques can effectively capture the complex, non-linear relationships in financial markets when coupled with rigorous data preprocessing, feature engineering, and systematic model evaluation.

Beyond technical success, the project has significant human, societal, ethical, and sustainability implications. By offering accessible, data-driven tools, it empowers individual investors to make informed financial decisions, reducing reliance on speculative trading. On a broader scale, more accurate forecasting can contribute to market stability and financial inclusivity, ensuring both large and small investors benefit. Ethically, transparent and fair model deployment protects vulnerable stakeholders and upholds data privacy standards, while the digital transformation of

trading practices supports sustainability through reduced resource consumption and the use of energy-efficient, green data centers.

Conclusion and Future Work

In conclusion, this project demonstrates the effective use of machine learning techniques for predicting daily stock closing prices based on historical market data. A well-structured pipeline was implemented that covered data collection, preprocessing, feature engineering, and model evaluation. Three regression algorithms—Support Vector Regression, Random Forest Regression, and Decision Tree Regression—were compared, with SVR proving to be the most consistent and reliable in terms of high R^2 scores on both training and validation sets. These results underscore the capability of machine learning models to capture complex market dynamics, while careful data handling and systematic feature selection played key roles in achieving robust predictive accuracy.

Looking ahead, there are promising avenues for enhancing this work further. Future developments could include the integration of real-time data streams from APIs like Alpha Vantage or Yahoo Finance, along with sentiment analysis from financial news and social media to capture market reactions more dynamically. Additionally, exploring advanced deep learning architectures such as LSTM networks or Transformer models could further improve forecasting accuracy, while interpretability tools like SHAP or LIME would enhance transparency and trust in prediction outcomes.

References

1. Cao, L. & Tay, F.E.H. (2003). **Support Vector Machine with Adaptive Parameters in Financial Time Series Forecasting.** IEEE Transactions on Neural Networks.

2. Patel, J., Shah, S., Thakkar, P., & Kotecha, K. (2015). **Predicting stock and stock price index movement using Trend Deterministic Data Preparation and machine learning techniques.** Expert Systems with Applications.

3. Brown, R.G. (1959). **Statistical Forecasting for Inventory Control.** McGraw/Hill.

4. Box, G.E.P. & Jenkins, G.M. (1970). **Time Series Analysis: Forecasting and Control.** Holden-Day.

5. Szrlee. (2018). **Stock Time Series 20050101 to 20171231 Dataset.** Kaggle. https://www.kaggle.com/datasets/szrlee/stock-time-series-20050101-to-20171231

6. Alpha Vantage API Documentation. https://www.alphavantage.co/documentation/

7. Neptune.ai Blog. (2021). **Predicting Stock Prices Using Machine Learning.** https://neptune.ai/blog/predicting-stock-prices-using-machine-learning

8. ScienceDirect. (2020). **Stock Market Prediction with High Accuracy using Machine Learning Techniques.** https://www.sciencedirect.com/science/article/pii/S1877050920307924

9. IEEE Xplore. (2019). **Stock Price Prediction Using News Sentiment Analysis.** https://ieeexplore.ieee.org/document/8848203

10. Yahoo Finance. https://finance.yahoo.com/